

GATE DA Question Practice Bank

Python | DSA | AI | DBMS — Verified PYQs, PYQ-Pattern & Original Questions

A note before you start:

GATE DA (Data Science & AI) has run only since 2024, so the genuine PYQ pool for narrow sub-topics (e.g. "two-pointer technique," "EER model," "BFS mechanics") is thin. Rather than fabricate fake PYQ years or question numbers, every question below is one of three clearly labeled types:

- **PYQ** — a genuine, verified past GATE question (exam, year, and question number cited)
- **PYQ-PATTERN** — closely modeled on a real, well-known exam pattern
- **ORIGINAL** — a new question written in authentic GATE style

Topics covered in this edition:

1. Python — Functions
2. Python — Recursion Basics
3. Python — Scope Rules
4. DSA — String Manipulation
5. DSA — Two Pointer Technique
6. AI — Problem Formulation
7. AI — State Space Representation
8. DBMS — EER Model
9. DBMS — ER-to-Relational Mapping
10. DBMS — Relational Model: Keys & Constraints
11. Python — Recursion Depth & Call-Stack Tracing
12. Python — Recursion Patterns (Factorial / Fibonacci / GCD) + Recurrence Writing
13. AI — Uninformed Search: BFS (Mechanics & Properties)

Each question includes a full solution, the concept tested, difficulty level, and question type. A compact answer key for every topic appears at the end of this document.

A note before you start: GATE DA (Data Science & AI) has only run since 2024, so the genuine PYQ pool for narrow sub-topics (e.g. "two-pointer technique," "EER model," "state space representation") is thin. Rather than fabricate fake PYQ years/numbers, I've done the following: - Included **verified genuine GATE DA 2024 PYQs** wherever they exist (checked against GATE Overflow), cited with exam/year/question number. - Everywhere else, built **PYQ-PATTERN** or **ORIGINAL** questions that mirror actual GATE DA/CSE question style, clearly labeled as such — never mislabeled as PYQ. - To keep every question fully verified and error-checked, each topic below has **8 high-quality questions** (rather than 20) — a mix of MCQ/MSQ/NAT. Tell me which topics you want expanded further and I'll add more in the same verified style.

1. PYTHON — Functions

Q1. What is the output of the following code?

```
def f(a, b=5, *args, **kwargs):
    return a + b + sum(args) + sum(kwargs.values())
print(f(1, 2, 3, 4, x=5, y=6))
```

Options: (A) 15 (B) 21 (C) 11 (D) TypeError

Answer: B

Solution: a=1, b=2 (positional overrides default), args=(3,4)→sum=7, kwargs={x:5,y:6}→sum=11. Total = 1+2+7+11 = 21.

Concept Tested: *args/**kwargs, positional vs default args | **Difficulty:** Easy | **Type:** ORIGINAL

Q2. What does the following print?

```
def add_item(item, lst=[]):
    lst.append(item)
    return lst
print(add_item(1))
print(add_item(2))
```

Options: (A) [1] then [2] (B) [1] then [1, 2] (C) [1, 2] then [1, 2] (D) Error

Answer: B

Solution: Default mutable arguments are created **once** at function definition, not per call. The same list object persists across calls, so it accumulates.

Concept Tested: Mutable default argument trap | **Difficulty:** Easy | **Type:** ORIGINAL

Q3. Which of the following calls raises a `TypeError`?

```
def f(a, b, *, c=10):
    return a*b + c
```

Options: (A) f(2,3) (B) f(2,3,c=5) (C) f(2,3,5) (D) f(a=2,b=3)

Answer: C

Solution: * forces everything after it to be keyword-only. `f(2, 3, 5)` passes 5 positionally for `c`, which is not allowed.

Concept Tested: Keyword-only arguments | **Difficulty:** Medium | **Type:** ORIGINAL

Q4. Consider:

```
def stats(lst):  
    return min(lst), max(lst), sum(lst)/len(lst)  
lo, hi, avg = stats([2, 4, 6, 8])
```

Which statements are TRUE? (MSQ) (A) `lo == 2` (B) `hi == 8` (C) `avg == 5.0` (D) The original list `lst` gets sorted in place

Answer: A, B, C

Solution: `min`, `max`, `sum/len` don't mutate the list; they only read values. `lo=2`, `hi=8`, `avg=(2+4+6+8)/4=5.0`. D is false — no in-place sort occurs.

Concept Tested: Multiple return values via tuple unpacking | **Difficulty:** Medium | **Type:** ORIGINAL

Q5. What is printed?

```
def outer():  
    count = 0  
    def inner():  
        nonlocal count  
        count += 1  
        return count  
    return inner  
c1 = outer()  
print(c1(), c1(), c1())
```

NAT Expected Answer: The three printed values are 1, 2, 3 → sum = 6

Solution: `inner` is a closure holding a reference to `count` in `outer`'s scope via `nonlocal`. Each call increments the same variable.

Concept Tested: Closures, nested functions, `nonlocal` | **Difficulty:** Medium | **Type:** ORIGINAL

Q6. What is printed?

```
def f(x):  
    if x > 0:  
        return x  
print(f(-3))
```

Options: (A) None (B) 0 (C) -3 (D) RuntimeError

Answer: A

Solution: If the `if` condition is false, the function falls off the end with no explicit `return`, so Python implicitly returns `None`.

Concept Tested: *Implicit return value* | **Difficulty:** *Medium* | **Type:** *ORIGINAL*

Q7. What is printed?

```
funcs = [lambda x: x + i for i in range(3)]
print([f(10) for f in funcs])
```

Options: (A) [10, 11, 12] (B) [12, 12, 12] (C) [10, 10, 10] (D) Error

Answer: B

Solution: Closures capture the **variable**, not its value at creation time. By the time the lambdas are called, the loop has finished and `i = 2` for all three, so every call returns `10+2 = 12`.

Concept Tested: *Late binding in closures — classic GATE-style conceptual trap* | **Difficulty:** *Hard* | **Type:** *ORIGINAL*

Q8. What is `len(a) + len(b)`?

```
def modify(lst):
    lst += [4]
    lst = lst + [5]
    return lst
a = [1, 2, 3]
b = modify(a)
print(len(a), len(b))
```

NAT Expected Answer: 9

Solution: `lst += [4]` is in-place extend (mutates the object `a` points to → `a` becomes `[1, 2, 3, 4]`, len 4). `lst = lst + [5]` creates a **new** list and rebinds the local name `lst`; `a` is unaffected. `b = [1, 2, 3, 4, 5]`, len 5. Total = 4+5 = 9.

Concept Tested: *Mutable object pass-by-reference vs rebinding* | **Difficulty:** *Hard* | **Type:** *ORIGINAL*

2. PYTHON — Recursion Basics

Q1 [PYQ]. *(GATE DA 2024, Question 28)*

```
def count(child_dict, i):
    if i not in child_dict.keys():
        return 1
    ans = 1
    for j in child_dict[i]:
        ans += count(child_dict, j)
    return ans

child_dict = dict()
child_dict[0] = [1, 2]
child_dict[1] = [3, 4, 5]
child_dict[2] = [6, 7, 8]
print(count(child_dict, 0))
```

Options: (A) 6 (B) 1 (C) 8 (D) 9

Answer: D

Solution: The function counts every node reachable from `i` (itself + all descendants), treating `child_dict` as a tree. Tree from 0: {0,1,2,3,4,5,6,7,8} → 9 nodes total.

Concept Tested: Recursive tree/graph traversal and counting | *Difficulty:* **Medium** | *Type:* **PYQ** (Exam: GATE DA, Year: 2024, Q28)

Q2 [PYQ]. *(GATE DA 2024, Question 31)*

```
def fun(D, s1, s2):
    if s1 < s2:
        D[s1], D[s2] = D[s2], D[s1]
        fun(D, s1+1, s2-1)
```

What does `fun()` do?

Options: (A) Finds the smallest element in D from s1 to s2 (B) Performs merge sort in-place between s1 and s2 (C) Reverses the list D between indices s1 and s2, inclusive (D) Swaps only D[s1] and D[s2], leaving the rest unchanged

Answer: C

Solution: Each recursive call swaps the outermost unprocessed pair and moves the pointers inward (`s1+1`, `s2-1`) until they cross — this is the classic two-pointer in-place reversal pattern applied recursively.

Concept Tested: Recursion simulating two-pointer swap (in-place reversal) | *Difficulty:* **Medium** | *Type:* **PYQ** (Exam: GATE DA, Year: 2024, Q31)

Q3. What is printed?

```
def fact(n):
    if n == 0:
        return 1
    return n * fact(n - 1)
print(fact(4))
```

Answer: 24 (Options: (A) 6 (B) 24 (C) 12 (D) 120)

Solution: $\text{fact}(4)=4\times\text{fact}(3)=4\times3\times\text{fact}(2)=4\times3\times2\times\text{fact}(1)=4\times3\times2\times1\times\text{fact}(0)=4\times3\times2\times1\times1=24$.

Concept Tested: Basic recursive trace | Difficulty: Easy | Type: ORIGINAL

Q4. What is the value of `fib(6)`?

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

NAT Expected Answer: 8

Solution: fib: 0,1,1,2,3,5,8 for $n=0..6$. $\text{fib}(6)=8$.

Concept Tested: Recursive Fibonacci trace | Difficulty: Easy | Type: ORIGINAL

Q5. What is printed?

```
def sum_digits(n):
    if n == 0:
        return 0
    return n % 10 + sum_digits(n // 10)
print(sum_digits(1234))
```

Options: (A) 10 (B) 1234 (C) 4 (D) 100

Answer: A

Solution: $1234\%10=4$, then $\text{sum_digits}(123)\rightarrow3+\text{sum_digits}(12)\rightarrow2+\text{sum_digits}(1)\rightarrow1+\text{sum_digits}(0)\rightarrow0$.
Total= $4+3+2+1=10$.

Concept Tested: Recursion on numeric digit extraction | Difficulty: Medium | Type: ORIGINAL

Q6. What is printed?

```
def is_even(n):
    if n == 0:
        return True
    return is_odd(n - 1)
def is_odd(n):
    if n == 0:
        return False
    return is_even(n - 1)
print(is_even(7))
```

Options: (A) True (B) False (C) None (D) RecursionError

Answer: B

Solution: Mutual recursion alternates between the two functions, decrementing n each time.

is_even(7)→is_odd(6)→is_even(5)→is_odd(4)→is_even(3)→is_odd(2)→is_even(1)→is_odd(0)→False.

Concept Tested: Mutual/indirect recursion | **Difficulty:** Medium | **Type:** ORIGINAL

Q7. What is the time complexity of the following recursive string reversal for a string of length n?

```
def rev(s):
    if len(s) <= 1:
        return s
    return rev(s[1:]) + s[0]
```

Options: (A) $O(n)$ (B) $O(n \log n)$ (C) $O(n^2)$ (D) $O(2^n)$

Answer: C

Solution: Each call does $O(k)$ work for slicing (`s[1:]`) and concatenation, over a recursion depth of n. Summing $O(n)+O(n-1)+\dots+O(1) = O(n^2)$. This is a common GATE-style complexity trap since strings are immutable.

Concept Tested: Recursion complexity with immutable string slicing | **Difficulty:** Hard | **Type:** ORIGINAL

Q8. What is printed?

```
def f(n):
    if n == 0:
        return 0
    return 1 + f(n - 1)
print(f(100))
```

NAT Expected Answer: 100

Solution: Each call adds 1 and recurses on n-1 until base case 0. This counts down from 100 to 0, accumulating 100 ones.

Concept Tested: Recursion depth / call-stack counting | **Difficulty:** Hard | **Type:** ORIGINAL

3. PYTHON — Scope Rules

(No genuine GATE DA PYQ specifically isolating "scope rules" was found in verified sources — all questions below are ORIGINAL/PYQ-PATTERN.)

Q1. What is printed?

```
x = 10
def f():
    x = 20
    return x
print(f(), x)
```

Options: (A) 20 20 (B) 10 10 (C) 20 10 (D) 10 20

Answer: C

Solution: Assignment inside `f` creates a new **local** `x` that shadows the global one; the global `x` is untouched.

Concept Tested: Local vs global shadowing | **Difficulty:** Easy | **Type:** ORIGINAL

Q2. What happens when `f()` is called?

```
x = 5
def f():
    print(x)
    x = 10
f()
```

Options: (A) Prints 5 (B) Prints 10 (C) UnboundLocalError (D) NameError

Answer: C

Solution: Because `x` is assigned anywhere inside `f`, Python treats `x` as **local to the entire function body** — even before the assignment line. The `print(x)` therefore references a local variable that hasn't been assigned yet.

Concept Tested: LEGB local-binding rule (classic trap) | **Difficulty:** Medium | **Type:** ORIGINAL

Q3. What is printed?

```
x = 5
def f():
    global x
    x = 10
f()
print(x)
```

Answer: 10

Solution: `global x` tells Python to use the module-level `x` instead of creating a local one, so the assignment modifies the global variable.

Concept Tested: `global` keyword | Difficulty: *Medium* | Type: *ORIGINAL*

Q4. Which of the following statements about Python scope are TRUE? (MSQ)

(A) Python resolves names using the Local → Enclosing → Global → Built-in (LEGB) order (B) The `global` keyword inside a nested function makes a name refer to its immediately enclosing function's scope (C) A variable assigned anywhere inside a function is treated as local for the entire function body, unless declared `global/nonlocal` (D) List comprehensions in Python 3 have their own local scope

Answer: A, C, D

Solution: B is false — `global` always refers to module-level scope; referring to an enclosing (non-global) function's scope requires `nonlocal`.

Concept Tested: LEGB rule, `global` vs `nonlocal` | Difficulty: *Medium* | Type: *ORIGINAL*

Q5. What is printed?

```
def outer():
    x = 1
    def inner():
        nonlocal x
        x += 1
    inner()
    inner()
    return x
print(outer())
```

Answer: 3

Solution: `nonlocal x` binds to `outer`'s `x`. Two calls to `inner()` each increment it by 1: 1→2→3.

Concept Tested: `nonlocal` keyword | Difficulty: *Medium* | Type: *ORIGINAL*

Q6. What is printed?

```
funcs = []
for i in range(3):
    def f():
        return i
    funcs.append(f)
print([f() for f in funcs])
```

Options: (A) [0, 1, 2] (B) [2, 2, 2] (C) [0, 0, 0] (D) Error

Answer: B

Solution: All three functions close over the **same** enclosing-scope variable `i`. By the time any `f()` is called, the loop has ended with `i = 2`.

Concept Tested: Closures + late binding (same trap as functions Q7, tested here from a scope-resolution angle) | **Difficulty:** *Hard* | **Type:** *ORIGINAL*

Q7. What is `len(lst)` after this code runs?

```
lst = [1, 2, 3]
def f():
    lst.append(4)
    f()
print(len(lst))
```

NAT Expected Answer: 4

Solution: `.append()` is a **mutation**, not a rebinding/assignment, so no `global` declaration is needed to modify the list contents from inside `f`.

Concept Tested: Mutating vs rebinding — common scope-rule misconception | **Difficulty:** *Hard* | **Type:** *ORIGINAL*

Q8. What is printed?

```
x = 'global'
class A:
    x = 'class'
    def method(self):
        return x
print(A().method())
```

Options: (A) global (B) class (C) AttributeError (D) NameError

Answer: A

Solution: Class bodies do **not** participate in the enclosing-scope lookup for methods (unlike nested functions). Inside `method`, `x` is looked up via LEGB, skipping the class namespace entirely, and resolves to the global `x`.

Concept Tested: Class scope is NOT part of LEGB — classic GATE-style conceptual trap | **Difficulty:** *Hard* | **Type:** *ORIGINAL*

4. DSA — String Manipulation

(No isolated genuine GATE DA/CSE PYQ matched this exact sub-topic in verified search; all questions are ORIGINAL/PYQ-PATTERN, styled after real GATE C/Python string-tracing questions.)

Q1. What is `s[2:6]` for `s = "GATEDA2026"`?

Options: (A) "TEDA" (B) "ATED" (C) "EDA2" (D) "TED"

Answer: A

Solution: Indices: G(0)A(1)T(2)E(3)D(4)A(5)2(6)... `s[2:6]` takes indices 2,3,4,5 → "TEDA".

Concept Tested: String slicing/indexing | **Difficulty:** Easy | **Type:** ORIGINAL

Q2. How many times does the character 's' occur in "mississippi"?

NAT Expected Answer: 4

Solution: m-i-s-s-i-s-i-p-p-i → 's' appears at positions 2,3,5,6.

Concept Tested: Character frequency counting | **Difficulty:** Easy | **Type:** ORIGINAL

Q3. What is printed?

```
def is_pal(s):  
    return s == s[::-1]  
print(is_pal("racecar"), is_pal("hello"))
```

Answer: True False

Solution: "racecar" reversed is itself → True. "hello" reversed is "olleh" ≠ "hello" → False.

Concept Tested: Palindrome check via slicing | **Difficulty:** Medium | **Type:** ORIGINAL

Q4. What happens when the following runs?

```
s = "hello"  
s[0] = 'H'
```

Options: (A) Prints "Hello" (B) TypeError: 'str' object does not support item assignment (C) IndexError (D) Silently does nothing

Answer: B

Solution: Strings in Python are **immutable**; individual characters cannot be reassigned. A new string must be built instead (e.g., `'H' + s[1:]`).

Concept Tested: String immutability trap | **Difficulty:** Medium | **Type:** ORIGINAL

Q5. Which of the following correctly determine whether two strings `s1`, `s2` are anagrams of each other? (MSQ)

(A) `sorted(s1) == sorted(s2)` (B) `Counter(s1) == Counter(s2)` (using `collections.Counter`) (C) `set(s1) == set(s2)` (D) `''.join(sorted(s1)) == ''.join(sorted(s2))`

Answer: A, B, D

Solution: C is incorrect — `set()` discards frequency/duplicate information, so `"aab"` and `"abb"` would wrongly be judged equal (`{'a', 'b'} == {'a', 'b'}`) despite not being anagrams.

Concept Tested: Anagram-check correctness, edge case with duplicate characters | **Difficulty:** Medium | **Type:** ORIGINAL

Q6. What is the time complexity of checking whether two strings of length `n` are anagrams using the sorting-based approach?

Options: (A) $O(n)$ (B) $O(n \log n)$ (C) $O(n^2)$ (D) $O(\log n)$

Answer: B

Solution: Sorting each string dominates the cost at $O(n \log n)$; the final comparison is $O(n)$.

Concept Tested: Complexity analysis of string algorithms | **Difficulty:** Hard | **Type:** ORIGINAL

Q7. Which statement about building a string of length `n` via repeated concatenation `s = s + ch` inside a loop is TRUE?

Options: (A) $O(n)$ overall, since strings support fast append (B) $O(n^2)$ overall, because each concatenation creates a new string object and copies existing content (C) $O(\log n)$ overall (D) $O(1)$ per operation, always

Answer: B

Solution: Because strings are immutable, each `s + ch` allocates a new string and copies all existing characters. Over n iterations this is $1+2+\dots+n = O(n^2)$. (Using a list + `''.join()` avoids this.)

Concept Tested: Immutability's impact on complexity — frequent GATE trap | **Difficulty:** Hard | **Type:** ORIGINAL

Q8. How many occurrences of the substring `"aa"` exist in `"aaaa"`, counting overlapping occurrences?

NAT Expected Answer: 3

Solution: Overlapping matches at positions (0,1), (1,2), (2,3) → `"aa", "aa", "aa"` → 3 occurrences.

Concept Tested: Overlapping substring counting, edge case | **Difficulty:** Hard | **Type:** ORIGINAL

5. DSA — Two Pointer Technique

(No genuine GATE DA/CSE PYQ isolating "two-pointer" as a named technique was found — all questions below are ORIGINAL/PYQ-PATTERN.)

Q1. Using the two-pointer technique on the sorted array [1, 2, 3, 4, 6] to find a pair summing to 6 (left=0, right=4, moving inward), which pair is found?

Options: (A) (1, 4) (B) (2, 4) (C) (1, 6) (D) (3, 3)

Answer: B

Solution: left=0, right=4: $1+6=7>6 \rightarrow \text{right--}$. left=0, right=3: $1+4=5<6 \rightarrow \text{left++}$. left=1, right=3: $2+4=6 \rightarrow \text{found (2,4)}$.

Concept Tested: Two-pointer pair-sum trace on sorted array | **Difficulty:** Easy | **Type:** ORIGINAL

Q2. In the trace from Q1, how many pointer comparisons are made before the pair is found (counting the successful one)?

NAT Expected Answer: 3

Solution: Comparisons: (0,4)→miss, (0,3)→miss, (1,3)→hit. Total = 3.

Concept Tested: Two-pointer step counting | **Difficulty:** Easy | **Type:** ORIGINAL

Q3. Using the standard slow/fast two-pointer technique to remove duplicates in-place from the sorted array [1, 1, 2, 2, 3], what is the length of the resulting unique-element prefix?

Options: (A) 2 (B) 3 (C) 4 (D) 5

Answer: B

Solution: Slow pointer advances only when a new distinct value is found: 1,2,3 → 3 unique elements.

Concept Tested: In-place duplicate removal via two-pointer | **Difficulty:** Medium | **Type:** ORIGINAL

Q4. Reversing a 6-element array in place using two pointers starting at both ends and swapping while moving inward requires how many swaps?

Options: (A) 2 (B) 3 (C) 4 (D) 6

Answer: B

Solution: For $n=6$, pointers meet after $\lceil 6/2 \rceil = 3$ swaps (indices (0,5),(1,4),(2,3)).

Concept Tested: In-place reversal, swap counting | **Difficulty:** Medium | **Type:** ORIGINAL

Q5. Which of the following statements about the two-pointer technique are TRUE? (MSQ)

(A) It generally requires the array to be sorted for pair-sum-style problems (B) On a sorted array, it reduces the pair-sum search from $O(n^2)$ brute force to $O(n)$ (C) It can be applied to any unsorted array for the sorted-duplicate-removal pattern without any preprocessing (D) It typically uses $O(1)$ extra space

Answer: A, B, D

Solution: C is false — the duplicate-removal-via-two-pointer pattern relies on the array being sorted so that duplicates are adjacent; on an unsorted array it would give wrong results without first sorting.

Concept Tested: Two-pointer applicability conditions | *Difficulty:* Medium | *Type:* ORIGINAL

Q6. Using two pointers on the sorted array `[1, 2, 3, 4, 5, 6, 7, 8]`, how many distinct pairs sum to exactly 9?

NAT Expected Answer: 4

Solution: Pairs: (1,8),(2,7),(3,6),(4,5) → 4 pairs.

Concept Tested: Two-pointer pair counting | *Difficulty:* Hard | *Type:* ORIGINAL

Q7. For the pair-sum problem on a sorted array of size n , which statement correctly compares two-pointer vs brute force?

Options: (A) Two-pointer: $O(n)$ time, $O(1)$ space; Brute force: $O(n^2)$ time, $O(1)$ space (B) Two-pointer: $O(n \log n)$ time; Brute force: $O(n)$ time (C) Both are $O(n^2)$ (D) Two-pointer needs $O(n)$ extra space

Answer: A

Solution: Two-pointer makes a single linear pass inward with no extra data structures; brute force checks all pairs.

Concept Tested: Complexity comparison | *Difficulty:* Hard | *Type:* ORIGINAL

Q8. For `arr = [5, 5, 5, 5]` and target sum = 10, using `left=0/right=3` two-pointer search, at which index pair is a valid pair immediately found?

Options: (A) (0, 1) (B) (0, 3) (C) (1, 2) (D) No pair found

Answer: B

Solution: `arr[0]+arr[3] = 5+5 = 10 = target` → found immediately on the first comparison. Edge case: all-equal-elements array still resolves correctly in one step.

Concept Tested: Two-pointer edge case with repeated elements | *Difficulty:* Hard | *Type:* ORIGINAL

6. AI — Problem Formulation

(AI is new to the GATE syllabus via GATE DA (2024 onward); no genuine PYQ on this specific sub-topic could be verified. All questions below are ORIGINAL/PYQ-PATTERN, modeled on classic problem-formulation scenarios — 8-puzzle, water-jug, missionaries-and-cannibals, 8-queens — that are standard in AI/GATE-style question banks.)

Q1. Which of the following is NOT a standard component of problem formulation in search-based AI?

Options: (A) Initial state (B) Goal test (C) Path cost function (D) Learning rate

Answer: D

Solution: The standard components are: initial state, actions/successor function, goal test, and path cost function. "Learning rate" belongs to machine learning, not classical search formulation.

Concept Tested: Components of problem formulation | **Difficulty:** Easy | **Type:** ORIGINAL

Q2. In the classic water-jug problem (a 4L jug and a 3L jug, goal: exactly 2L in the 4L jug), how is a state typically represented?

Options: (A) A single number (B) An ordered pair (x, y) — the current amount of water in each jug (C) A graph (D) A binary matrix

Answer: B

Solution: The state must capture all information relevant to the problem — here, just the water level in each jug — so (x, y) with $0 \leq x \leq 4$, $0 \leq y \leq 3$ suffices.

Concept Tested: State representation choice | **Difficulty:** Easy | **Type:** ORIGINAL

Q3. Which of the following statements about the 8-puzzle's problem formulation are TRUE? (MSQ)

(A) The branching factor at any state is at most 4 (up/down/left/right moves of the blank) (B) The total number of possible board arrangements is $9!$ (C) Only half of the $9!$ permutations are reachable from a given initial state, due to a solvability/parity constraint (D) The goal test result depends on the path taken to reach the current state

Answer: A, B, C

Solution: D is false — a goal test in a well-formulated problem depends only on the **current state**, never on the path used to reach it.

Concept Tested: State space properties, goal test definition | **Difficulty:** Medium | **Type:** ORIGINAL

Q4. In the classic Missionaries-and-Cannibals problem (3 missionaries, 3 cannibals, boat capacity 2, all starting on the left bank), what is the minimum number of boat crossings needed to get everyone across safely?

NAT Expected Answer: 11

Solution: This is the well-known minimum-crossing count for the standard 3M-3C, capacity-2 formulation of the problem.

Concept Tested: Problem formulation → solution path length | *Difficulty:* Medium | *Type:* PYQ-PATTERN

Q5. In formal problem formulation, an "operator" (action) is best defined as:

Options: (A) A function mapping a state to a resulting state (equivalently, a set of legal (state, action) → state transitions) (B) A number representing path cost (C) A test to check if the goal is reached (D) A heuristic estimate function

Answer: A

Solution: Actions/operators define the legal transitions between states; they are distinct from the goal test and cost function.

Concept Tested: Definitions: actions/operators | *Difficulty:* Medium | *Type:* ORIGINAL

Q6. Including excessive, irrelevant real-world detail in the state representation of a route-finding problem (e.g., tracking every passenger's mood) mainly leads to:

Options: (A) Faster search due to richer states (B) A smaller, more efficient state space (C) An unnecessarily large/intractable state space with no benefit to solving the problem (D) Guaranteed optimality of the solution

Answer: C

Solution: Good problem formulation abstracts away detail irrelevant to the goal; over-detailed states blow up the search space without helping find the solution.

Concept Tested: Abstraction in problem formulation | *Difficulty:* Hard | *Type:* ORIGINAL

Q7. A problem is said to be "well-defined" if its formulation specifies which of the following? (MSQ)

(A) Initial state (B) Successor function / set of actions (C) Goal test (D) Path cost function (E) The programming language used to implement the search

Answer: A, B, C, D

Solution: E is irrelevant — problem formulation is language-agnostic; it's a conceptual/mathematical specification.

Concept Tested: Well-defined problem — required components | *Difficulty:* Hard | *Type:* ORIGINAL

Q8. In the standard "complete-state" formulation of the 8-queens problem (each of 8 queens placed in any of the $8 \times 8 = 64$ squares, one queen per column, allowing any row 1–8), how many possible initial/candidate configurations exist before any constraint is applied?

NAT Expected Answer: 16777216 (i.e., 8^8)

Solution: With one queen fixed per column and 8 choices of row for each of the 8 columns, total configurations = 8^8
= 16,777,216.

Concept Tested: State space size calculation for a formulated problem | **Difficulty:** *Hard* | **Type:** *ORIGINAL*

7. AI — State Space Representation

(As with Problem Formulation, no verified genuine PYQ isolates this sub-topic — all questions below are ORIGINAL/PYQ-PATTERN.)

Q1. The "state space" of a search problem is best defined as:

Options: (A) The set of all possible actions (B) The set of all states reachable from the initial state via any sequence of legal actions (C) The goal state alone (D) The path cost function

Answer: B

Solution: The state space is the full set of states reachable by applying the successor function repeatedly, starting from the initial state.

Concept Tested: Definition of state space | **Difficulty:** Easy | **Type:** ORIGINAL

Q2. In a state-space graph representation of a search problem, nodes and edges correspond to:

Options: (A) Nodes = actions, Edges = states (B) Nodes = states, Edges = actions/operators connecting one state to another (C) Nodes = heuristics, Edges = costs (D) Nodes and edges both represent states

Answer: B

Solution: Standard convention: each node is a state; each directed edge represents an action transforming one state into another, often labeled with its cost.

Concept Tested: Graph-based state space representation | **Difficulty:** Easy | **Type:** ORIGINAL

Q3. For the standard 8-puzzle (3×3 grid), the blank tile has 2 possible moves at a corner position, 3 at an edge position, and 4 at the center position. If the blank is equally likely to be at any of the 9 cells, what is the average branching factor (rounded to two decimals)?

NAT Expected Answer: 2.67

Solution: $4 \text{ corners} \times 2 + 4 \text{ edges} \times 3 + 1 \text{ center} \times 4 = 8 + 12 + 4 = 24$. Average = $24/9 \approx 2.67$.

Concept Tested: Branching factor computation from state representation | **Difficulty:** Medium | **Type:** ORIGINAL

Q4. For the 8-queens problem, comparing "complete-state formulation" (all 8 queens placed at once, one per column) with "incremental formulation" (queens placed one at a time, left to right), which statement is TRUE?

Options: (A) Both have identical branching factor at every level (B) The incremental formulation typically has a much smaller effective branching factor at early levels, since only valid placements avoiding immediate attacks are considered as successors (C) The complete-state formulation always finds the goal faster (D) State representation

choice does not affect branching factor

Answer: B

Solution: Incremental formulation restricts successors to non-attacking placements early on, pruning the space compared to the complete-state formulation's massive successor set.

Concept Tested: Effect of representation choice on branching factor | *Difficulty:* Medium | *Type:* ORIGINAL

Q5. Which of the following are desirable properties of a good state-space representation? (MSQ)

(A) Captures all information relevant to solving the problem (B) Avoids unnecessary detail that inflates the state count without aiding the solution (C) Must always be represented as a numerical matrix (D) Allows the successor function to be computed efficiently

Answer: A, B, D

Solution: C is false — representation can be a tuple, graph, string, or any structure suited to the problem; a matrix is not mandatory.

Concept Tested: Desirable properties of state representation | *Difficulty:* Medium | *Type:* ORIGINAL

Q6. For the classic 15-puzzle (4×4 grid), the total number of distinct reachable states (accounting for the solvability parity constraint) is:

Options: (A) 16! (B) 16!/2 (C) 15! (D) 2^{16}

Answer: B

Solution: Of the 16! total permutations of tiles+blank, exactly half are reachable from any given solvable configuration due to the parity invariant of tile-swap permutations.

Concept Tested: State space size with solvability constraint | *Difficulty:* Hard | *Type:* ORIGINAL

Q7. Which statement correctly distinguishes explicit vs implicit state-space representation?

Options: (A) Explicit representation lists out all states in advance (e.g., as a stored graph); implicit representation generates states on-the-fly via a successor function as needed (B) Implicit representation is only usable for finite state spaces (C) Explicit representation is always more memory-efficient (D) There is no practical difference

Answer: A

Solution: Most real AI search problems (e.g., 8-puzzle, chess) use implicit representation since the full state space is too large to store explicitly; successors are generated as needed.

Concept Tested: Explicit vs implicit state space | *Difficulty:* Hard | *Type:* ORIGINAL

Q8. For a route-finding problem where cities are nodes and roads are edges with associated distances, this is an example of:

Options: (A) A graph-based explicit representation, suitable because the map is finite and known in advance (B) An implicit representation, since the map changes dynamically (C) A representation unsuitable for search algorithms (D) A continuous state space representation

Answer: A

Solution: Since the full map (all cities/roads) is known beforehand and finite, it can be explicitly represented as a weighted graph — unlike problems (e.g., robot motion planning) with continuous/unbounded state spaces.

Concept Tested: Choosing representation based on problem nature | **Difficulty:** Hard | **Type:** ORIGINAL

8. DBMS — EER Model

(EER is a classic, well-tested GATE CSE topic; however, no exact verbatim PYQ text/options could be confidently verified in this session without risking misquotation. All questions below are ORIGINAL/PYQ-PATTERN, built to match real GATE-style EER questions.)

Q1. In the EER model, "specialization" and "generalization" differ in that:

Options: (A) Specialization is top-down (superclass → subclasses); generalization is bottom-up (subclasses → superclass) (B) They are identical processes with different names (C) Specialization always requires disjoint subclasses (D) Generalization can only be applied to weak entities

Answer: A

Solution: Specialization identifies distinguishing subgroups within an entity set (top-down); generalization abstracts a common superclass from several entity sets sharing attributes (bottom-up).

Concept Tested: Specialization vs generalization | **Difficulty:** Easy | **Type:** ORIGINAL

Q2. If an entity instance of the superclass "Employee" can belong to at most one of the subclasses {Engineer, Manager, Technician}, this is an example of:

Options: (A) Overlapping constraint (B) Disjoint constraint (C) Total participation (D) Partial participation

Answer: B

Solution: "Disjoint" (shown as "d" in EER diagrams) means subclasses have no common members — each superclass entity belongs to at most one subclass.

Concept Tested: Disjoint vs overlapping constraints | **Difficulty:** Easy | **Type:** ORIGINAL

Q3. If every Employee entity must belong to at least one of {Engineer, Manager, Technician} (no employee can exist without being one of these), this specialization has:

Options: (A) Partial participation (B) Total participation (C) No participation constraint (D) A weak entity constraint

Answer: B

Solution: Total participation (double line in ER/EER diagrams) requires every superclass entity to have a corresponding subclass membership; partial allows some to have none.

Concept Tested: Total vs partial participation | **Difficulty:** Medium | **Type:** ORIGINAL

Q4. Which of the following statements about the EER model are TRUE? (MSQ)

(A) A subclass inherits all attributes and relationships of its superclass (B) A category (union type) can have subclasses coming from distinct, unrelated superclasses (e.g., Owner = Person $\cup$ Bank $\cup$ Company) (C) Disjoint and overlapping constraints can only be specified together with total participation, never with partial (D) Attribute-defined

specialization uses a defining attribute of the superclass to determine subclass membership

Answer: A, B, D

Solution: C is false — disjoint/overlapping and total/partial are independent, orthogonal constraints; all four combinations (disjoint-total, disjoint-partial, overlapping-total, overlapping-partial) are valid.

Concept Tested: EER inheritance, categories, constraint independence | **Difficulty:** Medium | **Type:** ORIGINAL

Q5. Out of 100 employees, 40 are Engineers, 35 are Managers, and 15 employees are both Engineer and Manager (overlapping specialization). How many employees belong to the union (Engineer $\cup$ Manager)?

NAT Expected Answer: 60

Solution: $|A \cup B| = |A| + |B| - |A \cap B| = 40 + 35 - 15 = 60$.

Concept Tested: Overlapping specialization — set-based reasoning | **Difficulty:** Medium | **Type:** ORIGINAL

Q6. A "category" (union type) in the EER model, such as `Owner = Person  $\cup$  Bank  $\cup$  Company` for a "Vehicle Owner" scenario, is used when:

Options: (A) A subclass needs to be defined for attributes shared identically by all superclasses (B) A single subclass relates to a union of instances from **multiple, generally unrelated** superclasses, unlike normal specialization where subclasses come from a single superclass (C) Only disjoint superclasses can ever form a category (D) Categories cannot have their own attributes

Answer: B

Solution: Categories model a subclass that is a subset of the union of distinct superclasses — a modeling construct not expressible through ordinary single-superclass specialization.

Concept Tested: Union type / category | **Difficulty:** Hard | **Type:** ORIGINAL

Q7. For an overlapping, total specialization of Employee into {Engineer, Manager}, the most common relational mapping approach uses:

Options: (A) One single table for Employee, Engineer, and Manager combined, with two extra boolean/type flag columns for subclass membership and NULLs for non-applicable subclass attributes (B) Three completely independent tables sharing no key (C) Deleting the superclass Employee table entirely (D) A single table with no way to represent subclass-specific attributes

Answer: A

Solution: For overlapping specializations, the common relational-mapping option (Option 7A / single-table-with-type-attributes technique) merges everything into one table with extra columns/flags to capture overlapping subclass membership.

Concept Tested: EER-to-relational mapping strategy choice for overlapping constraint | **Difficulty:** Hard | **Type:** ORIGINAL

Q8. A weak entity set "Dependent" exists only through the identifying relationship with strong entity "Employee," and additionally undergoes specialization into {Spouse, Child}. Which of the following is TRUE about mapping this to relational tables?

Options: (A) The weak entity's table must include Employee's primary key as part of its own primary key (partial key + owner PK), and this composite key must propagate down to Spouse/Child tables too if using separate subclass tables (B) Weak entities cannot be specialized (C) Spouse and Child tables need no foreign key reference back to Dependent/Employee (D) The weak entity does not need any foreign key at all

Answer: A

Solution: The weak entity's identifying dependency requires the owner's PK to be part of its composite key; this key (or its parent's referencing PK) must be carried into subclass tables to maintain the identification chain.

Concept Tested: Weak entity + specialization combined mapping | **Difficulty:** Hard | **Type:** ORIGINAL

9. DBMS — ER-to-Relational Mapping

(Also a classic, heavily-tested GATE CSE topic; questions below are ORIGINAL/PYQ-PATTERN, matching real GATE-style ER-mapping questions.)

Q1. For a 1:1 relationship between entities A and B where participation of A is total and B is partial, the most efficient mapping approach is:

Options: (A) Create a separate relationship table with FKs to both A and B (B) Merge B's primary key as a foreign key into A's table (the side with total participation), avoiding a separate table (C) Merge A's primary key as a foreign key into B's table (D) Always create three separate tables

Answer: B

Solution: For 1:1 relationships, placing the FK on the table with **total participation** avoids NULLs — every row in A will have a valid match, while some rows in B may not need the relationship.

Concept Tested: 1:1 relationship mapping, FK placement | **Difficulty:** Easy | **Type:** ORIGINAL

Q2. For an M:N relationship between entities A and B (with relationship attributes), relational mapping requires:

Options: (A) Adding B's key as FK into A's table (B) Adding A's key as FK into B's table (C) A separate junction/relationship table containing both A's and B's primary keys as a composite FK (and composite PK), plus any relationship attributes (D) No extra table is needed; use NULLs in both tables

Answer: C

Solution: M:N relationships cannot be captured by adding an FK to either side (would require multi-valued columns); a separate table with both keys as a composite primary/foreign key is required.

Concept Tested: M:N relationship mapping | **Difficulty:** Easy | **Type:** ORIGINAL

Q3. A weak entity set W depends on strong (owner) entity S via an identifying relationship. Correct relational mapping for W's table requires:

Options: (A) W's table needs only its own partial key as primary key (B) W's table's primary key = (partial key of W) + (primary key of S, as FK) (C) W does not need any reference to S at all (D) S's table absorbs W's attributes entirely

Answer: B

Solution: A weak entity has no independent primary key; its uniqueness comes from combining its partial key with the owner entity's primary key (also stored as an FK referencing S).

Concept Tested: Weak entity mapping | **Difficulty:** Medium | **Type:** ORIGINAL

Q4. An ER diagram has: 2 strong entities (Department, Employee) connected by a 1:N relationship "WorksIn," 1 weak entity (Dependent) owned by Employee, and 1 M:N relationship "Manages" between Employee and Project. How many relational tables are minimally needed (using standard mapping rules, no unnecessary merging)?

NAT Expected Answer: 5

Solution: Tables: Department, Employee, Dependent (weak entity table), Project, and a junction table for the M:N "Manages" relationship = 5 tables. The 1:N "WorksIn" relationship does NOT need a separate table — it's captured as an FK (Department's key) inside Employee's table.

Concept Tested: Counting tables from combined cardinalities (1:N, weak entity, M:N) | Difficulty: Medium | Type: ORIGINAL

Q5. Which of the following statements about mapping a multivalued attribute to a relational schema are TRUE? (MSQ)

(A) It requires creating a separate table containing the attribute value plus a foreign key referencing the owning entity's primary key (B) The owning entity's primary key + the multivalued attribute value together typically form the new table's primary key (C) It can always be stored as a single extra column in the owning entity's table without any issue (D) This mapping allows multiple values per entity instance without violating first normal form (1NF) at the relational level

Answer: A, B, D

Solution: C is false — cramming multiple values into one column of the owning entity's table would violate 1NF (atomic values); a separate table is the standard approach.

Concept Tested: Multivalued attribute mapping | Difficulty: Medium | Type: ORIGINAL

Q6. How is a composite attribute (e.g., Address = {Street, City, Pincode}) typically mapped to a relational schema?

Options: (A) A separate table is always created for it (B) Its simple component sub-attributes become separate columns directly within the owning entity's table; no separate table is generally needed (C) It is dropped entirely during mapping (D) It must be stored as a single unparsed string column

Answer: B

Solution: Composite attributes are typically "flattened" — each simple/atomic sub-attribute becomes its own column in the entity's table.

Concept Tested: Composite attribute mapping | Difficulty: Hard | Type: ORIGINAL

Q7. For a recursive (unary) M:N relationship, e.g., "Employee supervises Employee" (each employee can supervise many others and be supervised by many), the correct relational mapping is:

Options: (A) Add a single self-referencing FK column in the Employee table (B) A separate table with two FK columns, both referencing Employee's primary key, but under different role names (e.g., supervisor_id, supervisee_id) (C) No mapping is possible for recursive relationships (D) Duplicate the Employee table twice

Answer: B

Solution: Since it's M:N (not 1:N), a single self-referencing FK cannot capture the multiple links; a junction table with two differently-named FKs to the same entity's PK is required.

*Concept Tested: Recursive (unary) M:N relationship mapping | Difficulty: **Hard** | Type: ORIGINAL*

Q8. Three entities A, B, C are connected by three separate binary relationships, each 1:N: $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$ (the "many" side is always the second-named entity in each pair, e.g., one A relates to many B's). What is the minimum number of foreign key columns needed across all tables to represent these three relationships (using standard 1:N mapping, no extra junction tables)?

NAT Expected Answer: 3

Solution: Each 1:N relationship is mapped by placing one FK column on the "many" side's table, referencing the "one" side's PK: B gets an FK to A, C gets an FK to B, A gets an FK to C $\rightarrow$ 3 FK columns total, one per relationship.

*Concept Tested: FK counting across multiple 1:N relationships | Difficulty: **Hard** | Type: ORIGINAL*

10. DBMS — Relational Model: Keys & Constraints

(Classic GATE CSE topic; no exact verbatim PYQ text could be confidently re-verified in this session, so all questions below are ORIGINAL/PYQ-PATTERN, built to match real GATE-style key/constraint questions.)

Q1. Which of the following statements about keys in the relational model is TRUE?

Options: (A) A candidate key can contain NULL values (B) Every relation (as a proper set of tuples) must have at least one candidate key (C) A super key is always minimal (D) A foreign key must always be a candidate key of its own relation

Answer: B

Solution: Since a relation is a *set* of tuples, no two tuples can be identical, so the full attribute set (or some subset of it) always uniquely identifies every tuple — every relation has at least one candidate key. A is false (entity integrity/key uniqueness reasoning excludes NULL in candidate keys). C is false — super keys need not be minimal; a candidate key is a *minimal* super key. D is false — an FK references the PK of another (or the same) relation, not necessarily its own.

Concept Tested: Candidate key vs super key definitions | **Difficulty:** Easy | **Type:** ORIGINAL

Q2. A foreign key attribute FK in relation R2 references the primary key PK of relation R1. Under standard referential integrity, which statement is TRUE?

Options: (A) FK must always match an existing PK value in R1, with no exceptions (B) FK may be NULL (if it is not itself part of R2's primary key), or otherwise must match an existing PK value in R1 (C) FK must never be NULL under any circumstance (D) FK values must be unique within R2

Answer: B

Solution: Referential integrity requires that a non-NULL FK value must match an existing PK value in the referenced relation; NULL is permitted for the FK unless it is also part of R2's own primary key (in which case entity integrity forbids NULL).

Concept Tested: Referential integrity constraint | **Difficulty:** Easy | **Type:** ORIGINAL

Q3. For relation R(A, B, C, D) with functional dependency set $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$, what is the candidate key of R?

Options: (A) {A} (B) {A, B} (C) {A, D} (D) {B, C}

Answer: A

Solution: $A \rightarrow B \rightarrow C \rightarrow D$ by transitivity, so $A^+ = \{A, B, C, D\}$ = all attributes, and {A} is minimal (no proper subset — trivially just A itself). Hence {A} is the sole candidate key.

Concept Tested: Candidate key derivation from FDs | **Difficulty:** Medium | **Type:** ORIGINAL

Q4. For relation R(A, B, C, D, E) with FD set F = {AB→C, C→D, D→E}, which of the following statements are TRUE? (MSQ)

(A) AB is a candidate key of R (B) C is a candidate key of R (C) AB is the only candidate key of R (D) The closure of {A, B} under F equals all attributes of R

Answer: A, C, D

Solution: {A,B}⁺ = {A,B,C,D,E} = R, and neither A alone nor B alone appears on the left of any FD, so AB is minimal → candidate key. {C}⁺ = {C,D,E}, missing A,B → not a key. AB is therefore the unique candidate key.

Concept Tested: Closure computation, minimality of candidate keys | **Difficulty:** Medium | **Type:** ORIGINAL

Q5. In relation R(A, B, C, D), attribute A alone is the only candidate key (A determines B, C, D; no other combination not containing A is a key). How many total super keys does R have?

NAT Expected Answer: 8

Solution: Every super key must contain A. The remaining attributes {B,C,D} can each independently be included or excluded, giving $2^3 = 8$ possible super keys (all supersets of {A}).

Concept Tested: Counting super keys from a single candidate key | **Difficulty:** Medium | **Type:** ORIGINAL

Q6. The entity integrity constraint states that:

Options: (A) No attribute that is part of a primary key can contain a NULL value (B) Foreign key values must always reference an existing primary key value (C) All candidate keys across all relations in a schema must be globally unique (D) Composite attributes cannot be used as primary keys

Answer: A

Solution: Entity integrity specifically concerns NULL-freedom of primary key attributes; option B describes referential integrity instead, which is a distinct constraint.

Concept Tested: Entity integrity vs referential integrity | **Difficulty:** Hard | **Type:** ORIGINAL

Q7. R1(A,B) has primary key A. R2(C,D) has a composite primary key (C,D), where D is also a foreign key referencing R1.A. Can attribute D be NULL in a tuple of R2?

Options: (A) Yes, always allowed since D is a foreign key (B) No — since D is part of R2's primary key, entity integrity forbids NULL in it, overriding what referential integrity alone would permit (C) Only if C is also NULL (D) It depends solely on referential integrity, not entity integrity

Answer: B

Solution: Referential integrity alone would allow a non-key FK to be NULL, but here D is also part of the primary key of R2, so entity integrity takes precedence and forbids NULL regardless.

Concept Tested: Interaction between entity integrity and referential integrity | **Difficulty:** Hard | **Type:** ORIGINAL

Q8. A relation has 5 attributes and exactly two minimal (size-1) candidate keys, A and B, each independently determining all other attributes. How many total super keys does the relation have?

NAT Expected Answer: 24

Solution: Super keys containing A: $2^4 = 16$ (subsets of the remaining 4 attributes). Super keys containing B: 16. Super keys containing both A and B (double-counted): $2^3 = 8$ (subsets of the remaining 3 attributes). By inclusion-exclusion: $16 + 16 - 8 = 24$.

Concept Tested: Counting super keys with multiple candidate keys (inclusion-exclusion) | **Difficulty:** Hard | **Type:** ORIGINAL

11. PYTHON — Recursion Depth & Call-Stack Tracing

(No genuine PYQ verified for this narrow sub-topic in this session — all questions below are ORIGINAL/PYQ-PATTERN.)

Q1. How many stack frames are active at the deepest point while evaluating $f(5)$ below?

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

Answer: 6 (Options: (A) 5 (B) 6 (C) 4 (D) 10)

Solution: Calls are made for $n = 5, 4, 3, 2, 1, 0$ — that's 6 active frames at the deepest point (including the base case frame).

Concept Tested: Counting active stack frames | **Difficulty:** Easy | **Type:** ORIGINAL

Q2. For the same function f as in Q1, what is the maximum call-stack depth while evaluating $f(10)$?

NAT Expected Answer: 11

Solution: Calls for $n = 10$ down to 0 inclusive $\rightarrow$ 11 frames.

Concept Tested: Stack depth scaling with input | **Difficulty:** Easy | **Type:** ORIGINAL

Q3. What is the 4th line printed?

```
def f(n):  
    if n == 0:  
        print("base")  
        return  
    print("enter", n)  
    f(n - 1)  
    print("exit", n)  
f(3)
```

Options: (A) enter 1 (B) base (C) exit 1 (D) enter 2

Answer: B

Solution: Print order: "enter 3" (1st), "enter 2" (2nd), "enter 1" (3rd), "base" (4th), then unwinding prints "exit 1", "exit 2", "exit 3".

Concept Tested: Call-stack build-up and unwind order | **Difficulty:** Medium | **Type:** ORIGINAL

Q4. Which statement about recursion depth in Python is TRUE?

Options: (A) Python has no limit on recursion depth (B) The default recursion limit is typically 1000; exceeding it raises `RecursionError` (C) Recursion depth doesn't consume any memory (D) Tail recursion is automatically optimized by Python to avoid stack growth

Answer: B

Solution: Python's default `sys.getrecursionlimit()` is usually 1000. Unlike some functional languages, CPython performs **no** tail-call optimization (D is false), and every frame consumes stack memory (C is false).

Concept Tested: Python recursion limit, no TCO | Difficulty: Medium | Type: ORIGINAL

Q5. Which of the following statements about the call stack during recursion are TRUE? (MSQ)

(A) Each recursive call creates a new stack frame with its own local variables (B) Stack frames are removed (popped) as functions return, in LIFO order (C) All recursive calls share a single common set of local variables across all frames (D) Deeper recursion increases memory usage roughly proportional to depth

Answer: A, B, D

Solution: C is false — each frame has its own independent copy of local variables; that's precisely why recursion works correctly for tracking per-call state (e.g., each `n` in `f(n)` is distinct per frame).

Concept Tested: Stack frame independence and LIFO behavior | Difficulty: Medium | Type: ORIGINAL

Q6. What is the maximum simultaneous call-stack depth reached while evaluating `fib(5)` below (counting the initial call as depth 1)?

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

NAT Expected Answer: 5

Solution: The deepest path follows the leftmost branch: `fib(5)→fib(4)→fib(3)→fib(2)→fib(1)`, which is 5 nested active frames at once (the `fib(n-2)` branches are only explored after the deeper `fib(n-1)` branch has fully returned, so they never add extra simultaneous depth).

Concept Tested: Max stack depth in tree recursion (distinct from total call count) | Difficulty: Hard | Type: ORIGINAL

Q7. How many recursive call frames (including the initial call) are active when computing `gcd(48, 18)` below?

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)
```

Options: (A) 3 (B) 4 (C) 5 (D) 6

Answer: B

Solution: Trace: $\text{gcd}(48,18) \rightarrow \text{gcd}(18,12) \rightarrow \text{gcd}(12,6) \rightarrow \text{gcd}(6,0) \rightarrow$ returns 6. That's 4 calls total, hence 4 stack frames at the deepest point (this is tail recursion, but Python still stacks every frame).

Concept Tested: Recursive GCD trace and frame counting | **Difficulty:** Hard | **Type:** ORIGINAL

Q8. Which statement about tail-recursive functions in Python (such as the `gcd` function in Q7) is TRUE?

Options: (A) Python automatically converts tail recursion into iteration to save stack space (B) Even though the function is written in tail-recursive form, Python still creates a new stack frame for every call, since CPython lacks tail-call optimization (C) Tail recursion in Python never raises `RecursionError` (D) Tail recursion always runs faster in Python than an equivalent loop

Answer: B

Solution: Unlike languages such as Scheme, CPython does not perform tail-call optimization — every call, tail-recursive or not, adds a stack frame, so deep tail recursion can still hit the recursion limit.

Concept Tested: Absence of TCO in Python — common GATE-DA conceptual trap | **Difficulty:** Hard | **Type:** ORIGINAL

12. PYTHON — Recursion Patterns: Factorial, Fibonacci, GCD + Recurrence Writing

(All questions below are ORIGINAL/PYQ-PATTERN, covering the three canonical recursion patterns plus writing/solving their time-complexity recurrences — a frequently tested GATE-style combination.)

Q1. Which recurrence relation correctly describes the time complexity $T(n)$ of the recursive factorial function $\text{fact}(n) = n * \text{fact}(n-1)$, $\text{fact}(0) = 1$?

Options: (A) $T(n) = T(n-1) + O(1)$, $T(0) = O(1)$ (B) $T(n) = 2 \cdot T(n-1) + O(1)$ (C) $T(n) = T(n/2) + O(1)$ (D) $T(n) = T(n-1) + O(n)$

Answer: A

Solution: Each call does $O(1)$ work (one multiplication) besides the single recursive call on $n-1$, giving the linear recurrence $T(n) = T(n-1) + O(1)$, which solves to $T(n) = O(n)$.

Concept Tested: Writing the recurrence for simple linear recursion | **Difficulty:** Easy | **Type:** ORIGINAL

Q2. The naive recursive Fibonacci function $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ has the time-complexity recurrence $T(n) = T(n-1) + T(n-2) + O(1)$. What is the resulting asymptotic time complexity?

Options: (A) $O(n)$ (B) $O(n^2)$ (C) $O(2^n)$ (D) $O(n \log n)$

Answer: C

Solution: This recurrence has exponential growth (the tight bound is $O(\phi^n)$ where $\phi \approx 1.618$, commonly rounded up to $O(2^n)$ for GATE-level comparison), because of massive overlapping-subproblem recomputation with no memoization.

Concept Tested: Solving a recurrence for exponential-time recursion | **Difficulty:** Easy | **Type:** ORIGINAL

Q3. What is the time complexity of the recursive Euclidean algorithm $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ (base case $\text{gcd}(a, 0) = a$), in terms of the input values?

Options: (A) $O(a + b)$ (B) $O(\log(\min(a, b)))$ (C) $O(\min(a, b))$ (D) $O(a \cdot b)$

Answer: B

Solution: Each recursive step at least roughly halves one of the values (a known number-theoretic property of the mod operation), so the number of recursive calls is bounded by $O(\log(\min(a, b)))$; the worst case occurs for consecutive Fibonacci-number inputs.

Concept Tested: GCD recursion complexity | **Difficulty:** Medium | **Type:** ORIGINAL

Q4. What is $\text{fact}(6)$ using $\text{fact}(n) = n * \text{fact}(n-1)$, $\text{fact}(0) = 1$?

NAT Expected Answer: 720

Solution: $6 \times 5 \times 4 \times 3 \times 2 \times 1 \times 1 = 720$.

Concept Tested: Factorial recursion trace | **Difficulty:** Medium | **Type:** ORIGINAL

Q5. What is $\text{fib}(8)$ using $\text{fib}(0)=0$, $\text{fib}(1)=1$, $\text{fib}(n)=\text{fib}(n-1)+\text{fib}(n-2)$?

NAT Expected Answer: 21

Solution: Sequence: 0,1,1,2,3,5,8,13,21 for $n=0..8$. $\text{fib}(8)=21$.

Concept Tested: Fibonacci recursion trace | **Difficulty:** Medium | **Type:** ORIGINAL

Q6. How many recursive calls (including the initial call) does $\text{gcd}(150, 60)$ make using $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, base case $\text{gcd}(a, 0)=a$?

NAT Expected Answer: 3

Solution: Trace: $\text{gcd}(150, 60) \rightarrow \text{gcd}(60, 30) \rightarrow \text{gcd}(30, 0) \rightarrow$ returns 30. That's 3 calls total.

Concept Tested: GCD recursion trace / call counting | **Difficulty:** Medium | **Type:** ORIGINAL

Q7. Consider $\text{rsum}(n) = n + \text{rsum}(n-1)$, $\text{rsum}(0) = 0$ (recursive sum of first n natural numbers). Which recurrence and closed-form solution correctly describe its time complexity?

Options: (A) $T(n) = T(n-1) + O(1) \rightarrow O(n)$ (B) $T(n) = T(n-1) + O(n) \rightarrow O(n^2)$ (C) $T(n) = 2T(n-1) + O(1) \rightarrow O(2^n)$ (D) $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$

Answer: A

Solution: Just like factorial, one recursive call plus $O(1)$ extra work per level gives the linear recurrence $T(n)=T(n-1)+O(1)$, solving to $O(n)$.

Concept Tested: Recurrence writing for linear recursion pattern | **Difficulty:** Hard | **Type:** ORIGINAL

Q8. Which of the following statements comparing the factorial, Fibonacci, and GCD recursion patterns are TRUE? (MSQ)

(A) Factorial recursion and simple linear-sum recursion both satisfy $T(n)=T(n-1)+O(1)$, giving $O(n)$ time (B) Naive recursive Fibonacci has exponential time complexity due to repeated recomputation of overlapping subproblems (C) The recursive Euclidean GCD algorithm has logarithmic time complexity in the worst case (D) All three recursion patterns (factorial, Fibonacci, GCD) share the same asymptotic time complexity

Answer: A, B, C

Solution: D is false — the three patterns have distinct complexities: $O(n)$, $O(2^n)$, and $O(\log n)$ respectively, illustrating how the *shape* of the recurrence (single call vs. double call vs. shrinking-argument call) drives the asymptotic outcome.

Concept Tested: Comparative recurrence analysis across recursion patterns | **Difficulty:** Hard | **Type:** ORIGINAL

13. AI — Uninformed Search: BFS (Mechanics & Properties)

(No genuine PYQ verified for this exact sub-topic in this session — all questions below are ORIGINAL/PYQ-PATTERN, modeled on standard AI-search GATE-style questions.)

Q1. Which data structure does Breadth-First Search (BFS) use to manage its frontier (the set of nodes waiting to be expanded)?

Options: (A) Stack (LIFO) (B) Queue (FIFO) (C) Priority queue ordered by path cost (D) Two stacks

Answer: B

Solution: BFS always expands the shallowest (earliest-added) node first, which is exactly FIFO queue behavior.

Concept Tested: BFS frontier data structure | Difficulty: Easy | Type: ORIGINAL

Q2. On a graph/tree where every action has the same (uniform) step cost, which statement about BFS is TRUE?

Options: (A) BFS is complete but not optimal (B) BFS is both complete and optimal for finding the path with the fewest edges (C) BFS is optimal only if the graph happens to be a tree (D) BFS is neither complete nor optimal

Answer: B

Solution: With a finite branching factor and uniform step costs, BFS is guaranteed to find a solution if one exists (complete) and the shallowest one found is guaranteed shortest in edge-count (optimal).

Concept Tested: BFS completeness & optimality conditions | Difficulty: Easy | Type: ORIGINAL

Q3. For a search tree with uniform branching factor $b = 3$, if BFS explores completely down to and including depth $d = 3$ (a full tree, root at depth 0), what is the total number of nodes generated?

NAT Expected Answer: 40

Solution: Nodes at depth $k = b^k$. Total = $3^0 + 3^1 + 3^2 + 3^3 = 1 + 3 + 9 + 27 = 40$.

Concept Tested: Node-generation counting in BFS | Difficulty: Medium | Type: ORIGINAL

Q4. For branching factor b and solution depth d , what are BFS's time and space complexity (standard tree-search analysis)?

Options: (A) $O(b^d)$ time, $O(b^d)$ space (B) $O(b \cdot d)$ time, $O(d)$ space (C) $O(d^b)$ time, $O(b^d)$ space (D) $O(b^d)$ time, $O(d)$ space

Answer: A

Solution: BFS must generate all $O(b^d)$ nodes up to the solution depth, and — critically — must keep the entire frontier (which is itself $O(b^d)$ in size at the deepest fully-expanded level) in memory simultaneously.

Q5. Which of the following statements about BFS mechanics are TRUE? (MSQ)

(A) BFS always expands the shallowest unexpanded node first (B) BFS fully explores all nodes at depth k before expanding any node at depth $k+1$ (C) BFS generally uses far more memory than DFS, since it must store the entire frontier (which can be exponential in depth) (D) BFS always finds the optimal solution even when step costs are unequal

Answer: A, B, C

Solution: D is false — plain BFS is only optimal when all step costs are equal; for unequal costs, Uniform-Cost Search (not plain BFS) is needed to guarantee optimality.

Concept Tested: BFS properties and common misconception about weighted optimality | Difficulty: **Medium** | Type: **ORIGINAL**

Q6. Given a graph with edges S–A, S–B, A–C, B–C, C–D, and BFS starting at S visiting neighbors in alphabetical order, at what depth (level) is node D first discovered? (S is depth 0.)

NAT Expected Answer: 3

Solution: Depth 0: {S}. Depth 1: {A, B} (S's neighbors, alphabetical). Depth 2: from A → C is newly discovered (from B → C is already seen, skipped). Depth 3: from C → D is newly discovered. So D is first found at depth 3.

Concept Tested: BFS level-by-level traversal trace on a graph (with a shared neighbor) | Difficulty: **Hard** | Type: **ORIGINAL**

Q7. For a tree with branching factor b and depth d , which statement correctly compares the space complexity of BFS and DFS?

Options: (A) BFS: $O(b^d)$; DFS: $O(b \cdot d)$ — DFS is generally far more memory-efficient than BFS (B) Both have identical space complexity $O(b \cdot d)$ (C) DFS: $O(b^d)$; BFS: $O(b \cdot d)$ (D) Space complexity is the same regardless of strategy

Answer: A

Solution: DFS only needs to remember the single current path plus unexplored siblings at each level ($O(b \cdot d)$), while BFS must store the entire frontier, which grows exponentially with depth ($O(b^d)$).

Concept Tested: BFS vs DFS space complexity comparison | Difficulty: **Hard** | Type: **ORIGINAL**

Q8. For an unweighted graph (all edge costs equal), which uninformed strategy guarantees the same completeness/optimality as BFS but with better space complexity?

Options: (A) Plain DFS (B) Plain BFS itself is unbeatable on memory (C) Depth-limited search with an arbitrary limit (D) Iterative Deepening DFS (IDDFS) — same completeness/optimality guarantees as BFS, but $O(b \cdot d)$ space instead of $O(b^d)$

Answer: D

Solution: A common GATE-style trap: BFS is complete/optimal under uniform costs, but Iterative Deepening DFS achieves the identical guarantees while using only linear $O(b \cdot d)$ space, making it the generally preferred uninformed strategy when memory is a concern.

Concept Tested: Comparing uninformed strategies on completeness/optimality/space trade-offs | **Difficulty:** *Hard* | **Type:** *ORIGINAL*

ANSWER KEY (Compact)

#	Python-Fu nctions	Python-Re cursion	Python-Sc ope	DSA-Sim gs	DSA-Two Pointer	AI-ProbFor m	AI-StateSp ace	DBMS-EE R	DBMS-ER Map
1	B	D (9)	C	A	B	D	B	A	B
2	B	C (rev)	C	4 (NAT)	3 (NAT)	B	B	B	C
3	C	24	B(10)	True/False	B	A,B,C	2.67 (NAT)	B	B
4	A,B,C	8 (NAT)	A,C,D	B	B	11 (NAT)	B	A,B,D	5 (NAT)
5	6 (NAT)	A(10)	3	A,B,D	A,B,D	A	A,B,D	60 (NAT)	A,B,D
6	A	B(False)	B	B	4 (NAT)	C	B	B	B
7	B	C ($O(n^2)$)	4 (NAT)	B	A	A,B,C,D	A	A	B
8	9 (NAT)	100 (NAT)	A	3 (NAT)	B	16777216 (NAT)	A	A	3 (NAT)

#	DBMS-Keys	Recursion-Stack	Recursion-Patterns	AI-BFS
1	B	B(6)	A	B
2	B	11 (NAT)	C	B
3	A	B(base)	B	40 (NAT)
4	A,C,D	B	720 (NAT)	A
5	8 (NAT)	A,B,D	21 (NAT)	A,B,C
6	A	5 (NAT)	3 (NAT)	3 (NAT)
7	B	B(4)	A	A
8	24 (NAT)	B	A,B,C	D

Verified genuine PYQs used: GATE DA 2024 Q28 and Q31 (Python Recursion) — cross-checked against GATE Overflow's official answer keys.

Want more? I can expand any single topic to a full 20-question set (10 pattern-based + 10 original, properly ratio'd 2-easy/5-medium/3-hard) with the same verification rigor — just tell me which topic(s) to prioritize.